



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTY OF ENGINEERING

Databases

Group 1

Semester 2026-2

Final Project

Restaurant Operations Digitalization

Team Members:

Avila Sanchez Elias
Hernandez Flores Alan Uziel
Espinoza Gonzalez Sara Sofia
Palacio Amandi Fernando
Brena De Leon Victor Javier

Professor:

Ing. Fernando Arreola

May 31, 2026

Contents

1	Introduction	2
1.1	Problem Analysis	2
1.2	Objectives	2
1.3	Proposed Solution	2
2	Work Plan	2
2.1	Development Workflow	3
3	Design	3
3.1	Entity Identification and Business Rules	3
3.2	Entity-Relationship Model (ERM)	4
3.3	Relational Model Translation	5
4	Implementation	6
4.1	Human Resources Module	6
4.2	Menu Catalog and Operations Module	8
4.3	Data Integrity: Constraints and Sequences	9
4.4	Data Population (Data Manipulation Language - DML)	10
4.5	Programmability: Triggers, Views, and Functions	11
4.6	Part Two: Data Ingestion and Error Orchestration	14
5	Presentation	15
5.1	Workflow Design (Pipeline)	15
5.2	Engine Validations and Robustness (Error Handling)	15
6	Conclusions	16
6.1	Avila Sanchez Elias	16
6.2	Brena de León Víctor Javier	16
6.3	Espinosa González Sara Sofía	16
6.4	Hernández Flores Alan Uziel	17
6.5	Palacio Amandi Fernando	17

1 Introduction

1.1 Problem Analysis

The present project addresses the operational requirements of a restaurant seeking to digitize its daily operations. Currently, the establishment requires a robust information system to handle various operational modules, specifically focusing on the design and implementation of a centralized database. The core challenge involves managing complex entities and relationships, such as employee records with multiple possible roles (cooks, waiters, administrators), their dependents, a detailed catalog of dishes and beverages categorized by type, order tracking with exact calculations, and client billing information.

1.2 Objectives

The main objective of this project is to analyze the restaurant's requirements and propose a functional, optimized database solution applying the concepts learned throughout the course. Specific objectives include:

- Designing a database schema that accurately represents the business logic and satisfies all database design principles.
- Implementing the database utilizing PostgreSQL (PgSQL).
- Automating operations using tools such as views and indices to update totals, validate product availability, and generate invoice formats.
- Orchestrating the insertion of daily operational data from text files into the database with proper error handling and flow validation.

1.3 Proposed Solution

To solve the restaurant's digitalization needs, we propose a comprehensive relational database architecture in PostgreSQL. The solution will utilize an Entity-Relationship (ER) model and a Relational Model to establish strong data integrity across employees, products, orders, and clients. Furthermore, we will implement custom logic at the database level to dynamically update order totals and enforce inventory availability. Views will be constructed to quickly access critical business metrics, such as the best-selling dish and automated invoice generation. Finally, a robust data ingestion script will be developed to read and validate text files, ensuring seamless integration of the day's operations into the system.

2 Work Plan

The development of this project was carried out collaboratively. To ensure efficiency and maintain a high standard of quality in the final deliverables, tasks were strategically distributed among the team members. Each member took ownership of specific phases of the database lifecycle, from conceptual design to physical implementation and documentation.

The following table outlines the general activities and the primary responsibilities assigned to each member:

Team Member	Primary Responsibility
Avila Sanchez Elias	Document compilation, LaTeX formatting, and technical writing
Espinoza Gonzalez Sara Sofia	Conceptual design and Entity-Relationship Model (ERM) creation
Hernandez Flores Alan Uziel	Logical design, normalization, and Relational Model (RM) creation
Palacio Amandi Fernando	Database programming, DDL/DML scripting, and testing
Brena de Leon Victor Javier	Database programming, triggers, views, and advanced functions

Table 1: Task distribution among team members

2.1 Development Workflow

Instead of a rigid time-based schedule, our team adopted a sequential, phase-based workflow. This methodology allowed us to iterate on the design and ensure each stage was fully validated before moving to the next. The following flowchart illustrates the logical progression of the project’s lifecycle:

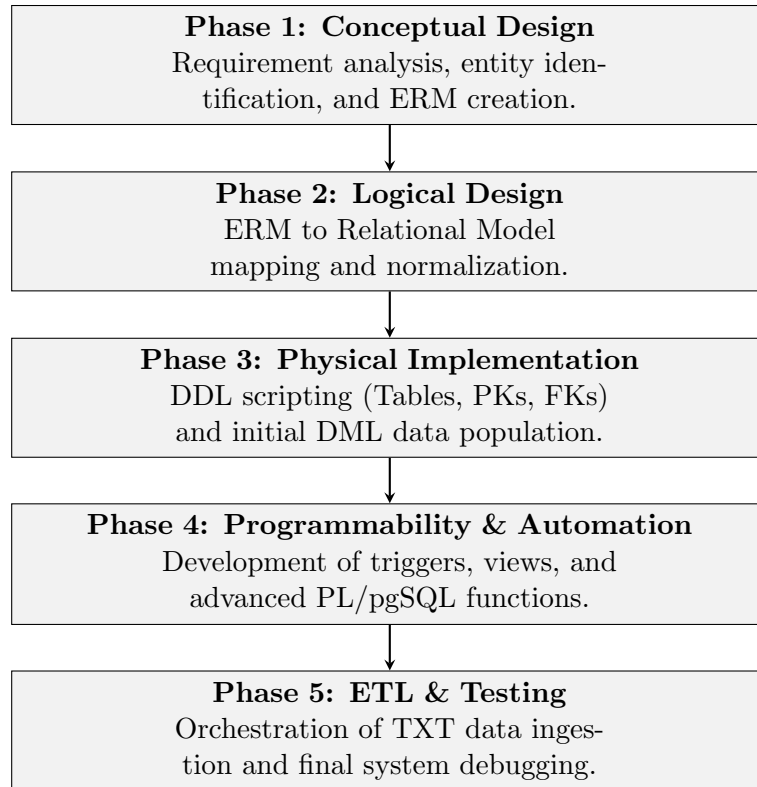


Figure 1: Sequential workflow of the database development lifecycle

3 Design

3.1 Entity Identification and Business Rules

The first phase of the database design involved analyzing the restaurant’s operational requirements to identify the core entities, their attributes, and the cardinalities of their relationships. The logical abstraction of the problem resulted in the following entity definitions:

- **Employee (EMPLEADO) and Subtypes:** This is a central entity that required an inheritance (hierarchy) model. Because the restaurant has different types of staff with specific attributes—cooks (COCINERO) with a specialty, waiters (MESERO) with a specific shift, and administrators (ADMINISTRATIVO) with a role—we established *EMPLEADO* as a supertype. General attributes like RFC, employee number (primary key), salary, and photograph belong to the supertype. To adhere to database design principles, attributes such as the employee’s name and address were modeled as composite attributes (e.g., the address is broken down into state, postal code, neighborhood, street, and number).
- **Dependent (DEPENDIENTE):** To keep track of an employee’s family members, we modeled this as a weak entity. A dependent cannot exist in the system without an associated employee. The relationship *tiene* (has) connects an Employee (1) to Dependents (M).
- **Category (CATEGORIA) and Product (PRODUCTO):** Products encompass all food and beverages offered by the restaurant. To maintain an organized menu, we identified a *CATEGORIA* entity. A 1:M relationship named *pertenece* (belongs to) was established, enforcing the rule that a product belongs to exactly one category, while a category can contain multiple products. Products store vital operational data such as price, recipe, and a boolean availability indicator.
- **Order (ORDEN):** This entity represents the core transaction of the restaurant. It is uniquely identified by a folio and records the date, time, and total amount payable. An order is intrinsically linked to the waiter (*MESERO*) who registered it through a 1:M relationship. To detail the items ordered, an M:N (many-to-many) relationship named *contiene* (contains) connects Orders and Products. The attributes “quantity” and “total per product” were placed directly on this relationship to accurately reflect the state of a specific order.
- **Client (CLIENTE):** To fulfill the requirement of issuing invoices, a *CLIENTE* entity was created, using the RFC as the primary key. This entity stores billing information, including a composite address and email. A 1:M relationship connects Clients to Orders, allowing a client to request invoices for multiple separate orders over time.

3.2 Entity-Relationship Model (ERM)

After defining the business rules and identifying the core entities, we constructed the Entity-Relationship Model to visually represent the database architecture and the interactions between all operational modules.

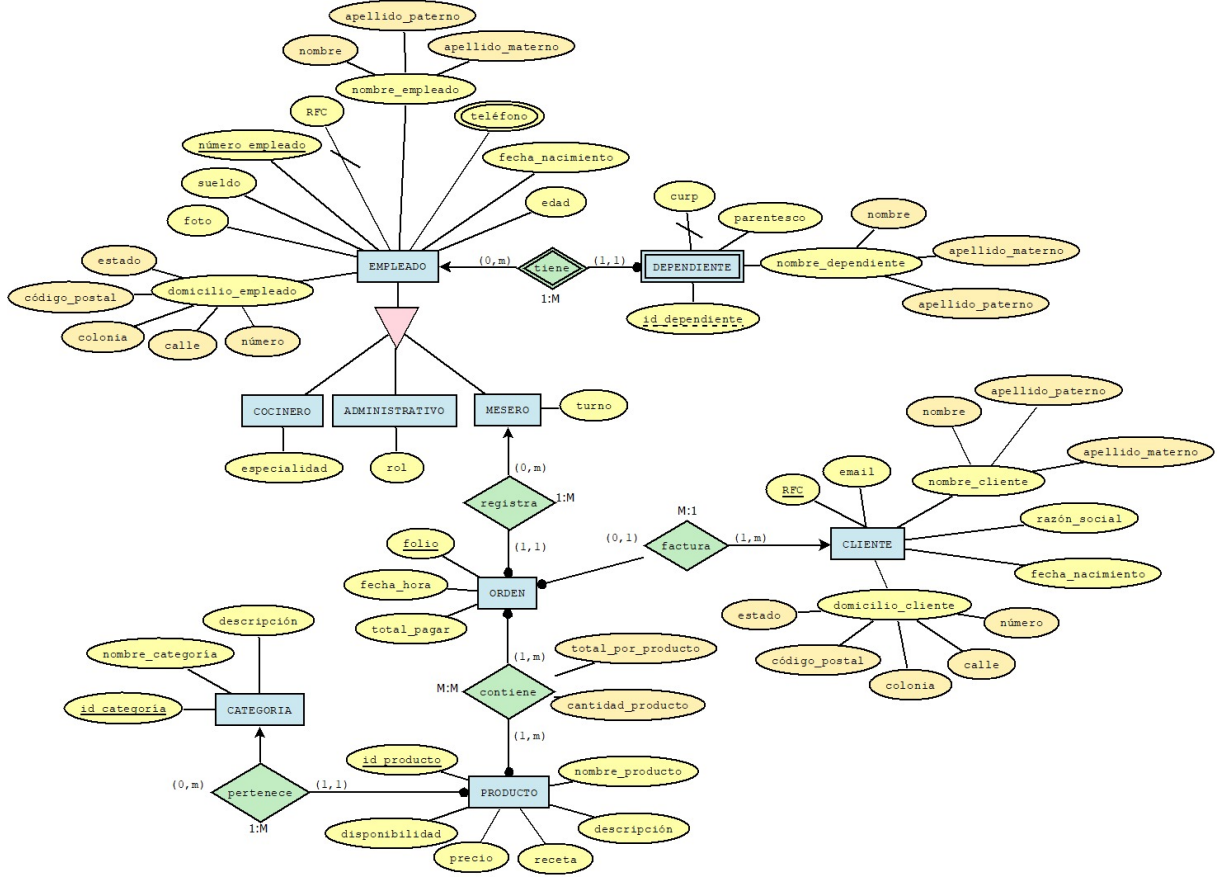


Figure 2: Entity-Relationship Model for the Restaurant Operations

3.3 Relational Model Translation

The transition from the Entity-Relationship Model to the Relational Model involved applying standard mapping rules to ensure structural integrity, normalization, and efficient data querying. The key structural transformations from the conceptual design to the relational schema include:

- **Composite and Multivalued Attributes:** Composite attributes, such as names and addresses for both employees and clients, were flattened into atomic columns directly within their respective tables (e.g., *estado_emp*, *codigo_postal_emp*, *colonia_emp*). The multivalued attribute for employee telephones required the creation of a new independent table, *EMPLEADO_TELEFONO*, linked to the main entity via the *numero_empleado* foreign key.
- **Hierarchy Implementation:** The employee supertype/subtype architecture was resolved using a class/subclass mapping strategy. The parent table *EMPLEADO* retains all generalized attributes and introduces binary discriminator fields (*es_cocinero*, *es_administrativo*, *es_mesero*). The specialized roles are represented as separate child tables (*COCINERO*, *ADMINISTRATIVO*, *MESERO*), where *numero_empleado* functions as both the primary key and a foreign key referencing the parent table.
- **Many-to-Many (M:N) Resolution:** The *contiene* relationship between *ORDEN* and *PRODUCTO* was transformed into an associative junction table named *ORDEN_PRODUCTO*.

This new table establishes a composite primary key consisting of the foreign keys *folio* and *id_producto*, while properly absorbing the transactional attributes *cantidad_producto* and *total_por_producto*.

- **One-to-Many (1:M) Relationships and Weak Entities:** Standard 1:M connections were mapped by placing foreign keys on the "many" side of the relationship. For example, *id_categoria* was integrated into PRODUCTO, and *rfc_cliente* along with the waiter's *numero_empleado* were added to ORDEN. For the weak entity DEPENDIENTE, the employee's primary key was imported as a foreign key to form an identifying relationship alongside its own identifier.

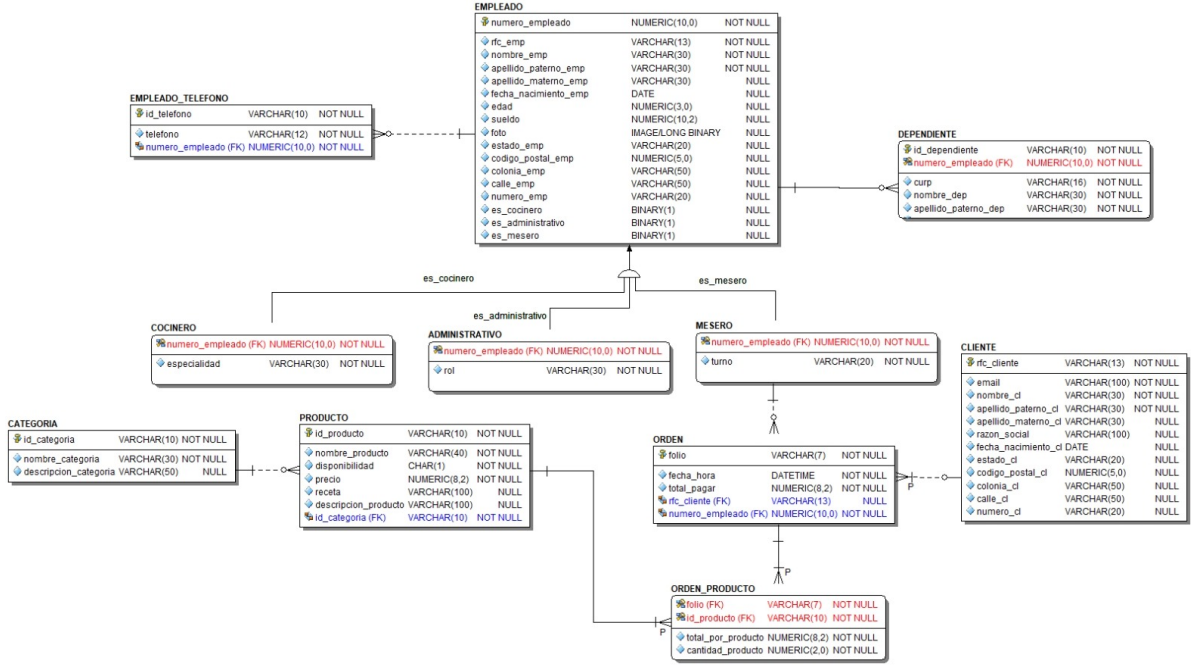


Figure 3: Relational Model derived from the ERM

4 Implementation

The implementation phase is a critical stage where the theoretical relational design is translated into a functional physical database using PostgreSQL (PgSQL). This process goes beyond merely creating tables; it involves making strategic decisions regarding data types, storage optimization, and the enforcement of business rules at the schema level. This section details the logical reasoning behind the Data Definition Language (DDL) scripts utilized to construct the database.

4.1 Human Resources Module

The first core block handles the logical mapping of the employee hierarchy, their contact information, and their dependents. To maintain document clarity, the following DDL snippets highlight only the structural constraints and routing attributes; the complete scripts are available in the repository.

```

1  -- Tabla Empleado (Supertype with discriminator flags)
2  CREATE TABLE EMPLEADO(
3      numero_empleado      numeric(10, 0)      NOT NULL,
4      rfc_emp              varchar(13)          NOT NULL,
5      /* ... other general attributes (name, address, DOB) ... */
6      es_cocinero           boolean             NULL,
7      es_administrativo     boolean             NULL,
8      es_mesero             boolean             NULL,
9      CONSTRAINT PK1 PRIMARY KEY (numero_empleado)
10 );
11
12 -- Subtypes (Specialized roles sharing the PK)
13 CREATE TABLE COCINERO(
14     numero_empleado      numeric(10, 0)      NOT NULL,
15     especialidad          varchar(30)         NOT NULL,
16     CONSTRAINT PK4 PRIMARY KEY (numero_empleado)
17 );
18 /* CREATE TABLE ADMINISTRATIVO (...) */
19 /* CREATE TABLE MESERO (...) */
20
21 -- Multivalued Attributes and Weak Entities
22 CREATE TABLE EMPLEADO_TELEFONO(
23     id_telefono           varchar(10)         NOT NULL,
24     telefono              varchar(12)         NOT NULL,
25     numero_empleado      numeric(10, 0)      NOT NULL,
26     CONSTRAINT PK2 PRIMARY KEY (id_telefono)
27 );
28
29 CREATE TABLE DEPENDIENTE(
30     id_dependiente        varchar(10)         NOT NULL,
31     numero_empleado      numeric(10, 0)      NOT NULL,
32     /* ... other dependent attributes (name, curp) ... */
33     CONSTRAINT PK3 PRIMARY KEY (id_dependiente, numero_empleado)
34 );

```

Listing 1: DDL for Employee Hierarchy, Telephones, and Dependents (Snippet)

Implementation Logic: The architecture of the `EMPLEADO` table demonstrates the application of the Class/Subclass mapping strategy. To fulfill the specific business rule that states “an employee can hold multiple roles simultaneously,” we implemented a generalized supertype table (`EMPLEADO`) coupled with boolean-like discriminator attributes (`es_cocinero`, `es_administrativo`, `es_mesero`). This structure allows a single employee record to be flagged for multiple operations without data redundancy. The specialized attributes of each role are isolated in their respective subtype tables (`COCINERO`, `ADMINISTRATIVO`, `MESERO`), which share the `numero_empleado` as both a Primary Key and a Foreign Key.

Regarding data types, `NUMERIC(10,0)` was chosen for the `numero_empleado` rather than an integer type to guarantee exact numerical precision and prevent potential overflow issues in highly scalable systems. For string attributes, `VARCHAR` is utilized over `CHAR` for names and addresses to optimize disk space by only storing the actual string length, whereas `CHAR` is reserved strictly for fixed-length data like the boolean discriminators. Furthermore, to maintain atomic values and adhere to First Normal Form (1NF), the

multivalued telephone attribute was extracted into the EMPLEADO_TELEFONO table, establishing a composite key structure.

4.2 Menu Catalog and Operations Module

This module governs the business logic surrounding the products offered and the transactions (orders) made by clients, focusing heavily on transactional integrity.

```
1  -- Catalog (Strong typing for monetary values)
2  /* CREATE TABLE CATEGORIA (...) */
3
4  CREATE TABLE PRODUCTO(
5      id_producto          varchar(10)          NOT NULL,
6      /* ... catalog attributes (name, recipe, category) ... */
7      disponibilidad       char(1)              NOT NULL,
8      precio               numeric(8, 2)        NOT NULL,
9      CONSTRAINT PK8 PRIMARY KEY (id_producto)
10 );
11
12 -- Clients and Orders (Transactions and M:N resolution)
13 /* CREATE TABLE CLIENTE (...) */
14
15 CREATE TABLE ORDEN(
16     folio                 varchar(7)           NOT NULL,
17     fecha_hora            timestamp            NOT NULL,
18     total_pagar           numeric(8, 2)        NOT NULL,
19     /* ... foreign keys (rfc_cliente, numero_empleado) ... */
20     CONSTRAINT PK10 PRIMARY KEY (folio)
21 );
22
23 CREATE TABLE ORDEN_PRODUCTO(
24     folio                 varchar(7)           NOT NULL,
25     id_producto           varchar(10)          NOT NULL,
26     total_por_producto    numeric(8, 2)        NOT NULL,
27     cantidad_producto     numeric(2, 0)        NOT NULL,
28     CONSTRAINT PK14 PRIMARY KEY (folio, id_producto)
29 );
```

Listing 2: DDL for Categories, Products, Clients, and Orders (Snippet)

Implementation Logic: The core of the transactional system lies in the resolution of the Many-to-Many (M:N) relationship between `ORDEN` and `PRODUCTO`. In relational theory, this must be resolved using an associative entity, implemented here as the `ORDEN_PRODUCTO` table. The composite primary key (*folio*, *id_producto*) guarantees that a specific product is only registered once per order line.

Crucially, the attributes *cantidad_producto* (quantity) and *total_por_producto* (total per product) are placed within this associative table. This is a fundamental database design principle: these attributes represent the state of the product *at the exact moment* of the transaction. If the price of a product changes in the `PRODUCTO` catalog tomorrow, historical orders must remain unaffected.

For monetary values (*precio*, *total_pagar*), the `NUMERIC(8,2)` data type is strictly enforced. Unlike floating-point types (`REAL` or `DOUBLE`), `NUMERIC` performs exact arithmetic,

which is mandatory for financial calculations to avoid rounding anomalies. The `ORDEN` table utilizes the `timestamp` data type to record both the date and the precise time the waiter registered the transaction, satisfying the specific project requirement.

4.3 Data Integrity: Constraints and Sequences

To guarantee database consistency and meet specific formatting requirements, `ALTER TABLE` statements were executed to enforce declarative referential integrity.

```
1 -- Unique Constraints
2 ALTER TABLE EMPLEADO ADD CONSTRAINT UQ_empleado_rfc UNIQUE (
   rfc_emp);
3 ALTER TABLE DEPENDIENTE ADD CONSTRAINT UQ_curp_dependiente UNIQUE
   (curp);
4 ALTER TABLE CLIENTE ADD CONSTRAINT UQ_cliente_email UNIQUE (email
   );
5
6 -- Sequences for Custom ID Generation
7 CREATE SEQUENCE seq_id_dependiente START WITH 1;
8 ALTER TABLE DEPENDIENTE
9 ALTER COLUMN id_dependiente SET DEFAULT ('DEP' || LPAD(NEXTVAL('
   seq_id_dependiente')::TEXT, 2, '0'));
10
11 CREATE SEQUENCE seq_folio_orden START WITH 1;
12 ALTER TABLE ORDEN
13 ALTER COLUMN folio SET DEFAULT ('ORD-' || LPAD(NEXTVAL('
   seq_folio_orden')::TEXT, 3, '0'));
14
15 -- Example of Foreign Key assignment (Order to Waiter and Client)
16 ALTER TABLE ORDEN ADD CONSTRAINT FK_Orden_Cliente
17 FOREIGN KEY (rfc_cliente) REFERENCES CLIENTE(rfc_cliente);
18 ALTER TABLE ORDEN ADD CONSTRAINT FK_Orden_Mesero
19 FOREIGN KEY (numero_empleado) REFERENCES MESERO(
   numero_empleado);
```

Listing 3: Referential Integrity and Sequence Automation

Implementation Logic: A robust database must protect its own integrity regardless of the application interacting with it. By declaring `FOREIGN KEY` constraints at the schema level, the database natively prevents orphan records (e.g., preventing an order from being registered to a non-existent waiter or client). `UNIQUE` constraints were strategically applied to natural keys such as the RFC, CURP, and client emails to prevent accidental data duplication.

One of the most technical implementations in this module is the fulfillment of the required order folio format (e.g., "ORD-001"). Rather than calculating this format at the application level, which is prone to concurrency issues, we delegated this responsibility to the database engine using custom `SEQUENCE` objects. By utilizing the `NEXTVAL()` function to retrieve the next integer, casting it to text (`::TEXT`), applying the `LPAD()` function to ensure a strict three-digit zero-padded format, and finally concatenating (`||`) it with the "ORD-" prefix, the database automatically and safely generates properly formatted, concurrent-safe primary keys.

4.4 Data Population (Data Manipulation Language - DML)

Now that the physical schema is built, we needed to populate the database with a robust set of test data. Instead of just adding a couple of random rows, we generated enough records (around 15 to 20 per main table) to properly test our future queries, views, and automated triggers. We carefully structured our INSERT statements to respect the referential integrity (Foreign Keys) defined in the previous phase.

1. Populating the Employee Hierarchy

Because of our supertype/subtype architecture, the order of insertion is crucial. We cannot register a waiter without first registering them as a general employee.

```
1 -- 1. Insert into the Supertype first
2 INSERT INTO EMPLEADO (numero_empleado, rfc_emp, nombre_emp,
3     apellido_paterno_emp, es_cocinero, es_administrativo, es_mesero
4     , ...)
5 VALUES (1000000004, 'GARM981115ABC', 'Mariana', 'Garcia', false,
6     false, true, ...);
7
8 -- 2. Insert into the Subtype using the same PK/FK
9 INSERT INTO MESERO (numero_empleado, turno)
10 VALUES (1000000004, 'Matutino');
11
12 -- 3. Insert dependent entities
13 INSERT INTO EMPLEADO_TELEFONO (id_telefono, telefono,
14     numero_empleado)
15 VALUES ('TEL-04', '5532109876', 1000000004);
```

Listing 4: Snippet of DML for the Employee Hierarchy

Implementation Logic: As shown in the snippet, we first insert the record into the EMPLEADO table, making sure to correctly set the boolean discriminator flags (e.g., *es_mesero* = *true*). Immediately after, we use that exact same *numero_empleado* to insert the specific shift information into the MESERO table. If we tried to reverse this order, the database engine would immediately block the transaction due to a Foreign Key violation. This proves our inheritance model is solidly linked at the data level.

2. Menu Catalog and Clients

For the operational catalog, Categories had to be inserted before Products so that the products had a valid ID to reference.

```
1 INSERT INTO CATEGORIA (id_categoria, nombre_categoria) VALUES ('
2     CAT-02', 'Platillos Fuertes');
3
4 INSERT INTO PRODUCTO (id_producto, nombre_producto,
5     disponibilidad, precio, id_categoria, ...)
6 VALUES ('PROD-02', 'Ramen de Shoyu', 'S', 140.00, 'CAT-02', ...),
7     ('PROD-11', 'Yakimeshi Especial', 'N', 110.00, 'CAT-02',
8     ...);
9
10 INSERT INTO CLIENTE (rfc_cliente, email, nombre_cl, ...)
11 VALUES ('XAXX010101000', 'publico@general.com', 'Publico', ...);
```

Listing 5: Snippet of DML for Catalog and Clients

Implementation Logic: A key detail in our testing strategy was deliberately inserting products with a availability status of 'N' (like the Yakimeshi Especial). This is necessary to verify that our future validation triggers actually work and prevent out-of-stock items from being ordered. Additionally, for the `CLIENTE` table, we included a generic RFC record (XAXX010101000) for "Público General". This reflects real-world business logic: not every restaurant customer requests an invoice, but the system still needs a valid client entity to link the daily sales.

3. Transactional Data (Orders)

Finally, we populated the core operations of the restaurant.

```
1 -- Registering the Order header
2 INSERT INTO ORDEN (folio, fecha_hora, total_pagar, rfc_cliente,
3   numero_empleado)
4 VALUES ('ORD-001', '2026-05-01 12:30:00', 335.00, 'GOMA850315HJK',
5   1000000004);
6
7 -- Registering the items inside the order
8 INSERT INTO ORDEN_PRODUCTO (folio, id_producto,
9   total_por_producto, cantidad_producto)
10 VALUES ('ORD-001', 'PROD-02', 280.00, 2), -- 2 Ramen @ 140 each
11         ('ORD-001', 'PROD-03', 55.00, 1); -- 1 Mochi @ 55 each
```

Listing 6: Snippet of DML for Orders and Order Details

Implementation Logic: When filling the `ORDEN_PRODUCTO` associative table, we made sure the math actually adds up based on the prices defined in the `PRODUCTO` table. For instance, if a Ramen costs \$140.00 and the quantity is 2, the *total_por_producto* is correctly recorded as \$280.00. While the project requirements state that these totals will be updated automatically via triggers later on, having a consistent, mathematically sound baseline of test data is critical to verify that our views and historical queries calculate revenue accurately.

4.5 Programmability: Triggers, Views, and Functions

To satisfy the specific operational requirements, we implemented active business logic directly within the database engine using PL/pgSQL. This approach ensures that rules are universally enforced regardless of the front-end application connecting to the system.

1. Business Rules Automation (Triggers)

A critical requirement was to automatically update order totals and validate product availability whenever a new item is added to an order. We achieved this through a `BEFORE INSERT` trigger.

```
1 CREATE OR REPLACE FUNCTION fn_validar_producto_orden()
2 RETURNS TRIGGER AS $$
3 DECLARE
4   v_disponibilidad CHAR(1);
5   v_precio NUMERIC(8,2);
6 BEGIN
7   SELECT disponibilidad, precio INTO v_disponibilidad, v_precio
8   FROM PRODUCTO WHERE id_producto = NEW.id_producto;
9
```

```

10     IF v_disponibilidad <> 'S' THEN
11         RAISE EXCEPTION 'El producto % no esta disponible.', NEW.
            id_producto;
12     END IF;
13
14     -- Calculate the total per product and overwrite the incoming
        INSERT value
15     NEW.total_por_producto := v_precio * NEW.cantidad_producto;
16
17     -- Update the grand total in the parent table (ORDEN)
18     UPDATE ORDEN SET total_pagar = total_pagar + NEW.
        total_por_producto
19     WHERE folio = NEW.folio;
20
21     RETURN NEW;
22 END;
23 $$ LANGUAGE plpgsql;
24
25 CREATE OR REPLACE TRIGGER trg_antes_agregar_producto
26 BEFORE INSERT ON ORDEN_PRODUCTO
27 FOR EACH ROW EXECUTE FUNCTION fn_validar_producto_orden();

```

Listing 7: Trigger for Product Validation and Total Calculation

Implementation Logic: We specifically chose a **BEFORE** trigger rather than an **AFTER** trigger. This allows us to intercept the incoming record (using the **NEW** record variable), perform the mathematical calculation ($\text{precio} \times \text{cantidad}$), and overwrite **NEW.total_por_producto** before it is permanently written to disk. Simultaneously, the function dynamically executes an **UPDATE** statement on the parent **ORDEN** table, keeping the grand total perfectly synchronized with the order’s contents.

2. Performance Optimization (Indexing)

```

1 CREATE INDEX idx_orden_fecha_hora ON ORDEN (fecha_hora);

```

Listing 8: B-Tree Index Creation

Implementation Logic: The **ORDEN** table represents the heaviest transactional growth in the system. Because multiple reporting functions (such as the daily sales summary and custom date-range reports) rely heavily on filtering by the *fecha_hora* column, an index was mandatory. Without it, PostgreSQL is forced to execute a Sequential Scan (Seq Scan), evaluating every row in the table, resulting in a linear $O(N)$ time complexity. By explicitly creating a B-Tree index, the database organizes these timestamps into a balanced tree structure. This allows the query planner to perform extremely fast Index Scans, reducing the search complexity to $O(\log N)$, which is specifically optimized for range operators like **BETWEEN**.

3. Reporting and Data Abstraction (Views and Functions)

To facilitate data extraction without requiring end-users to write complex SQL, we encapsulated the logic into specialized Views and Functions whose code you can find on GitHub.

- **Data Validation in Functions:** For the requirement of tracking daily orders

per employee, we created `fn_ordenes_del_dia`. Before performing the count, this function executes a validation query against the `MESERO` subtype table. If the ID does not belong to a waiter, it intentionally halts execution using `RAISE EXCEPTION`, strictly enforcing the business rule that only waiters can manage orders.

- **Complex Abstraction (Invoice View):** We developed `vista_factura` to generate a complete invoice format. This view abstracts a massive multi-table JOIN operation across `ORDEN`, `CLIENTE`, `EMPLEADO`, `ORDEN_PRODUCTO`, and `PRODUCTO`. It completely shields the application layer from the underlying normalized schema, presenting a single, flattened virtual table containing all necessary fiscal and detail data.

4. Operational Queries and Aggregations

To fully cover the analytical needs without cluttering the document with exhaustive SQL scripts (which are available in their entirety in the attached GitHub repository), the following snippets highlight the core logic of our reporting functions:

```

1  -- Top selling product (Core Subquery Logic)
2  CREATE OR REPLACE VIEW vista_platillo_mas_vendido AS
3      SELECT p.id_producto, p.nombre_producto, /*...*/ ventas.
         total_vendido
4      FROM PRODUCTO p /*...*/
5      JOIN (
6          SELECT id_producto, SUM(cantidad_producto) AS
             total_vendido
7          FROM ORDEN_PRODUCTO GROUP BY id_producto
8          ORDER BY total_vendido DESC LIMIT 1
9      ) AS ventas ON ventas.id_producto = p.id_producto;
10
11 -- Unavailable products filter
12 CREATE OR REPLACE FUNCTION fn_productos_no_disponibles() /*...*/
13     RETURN QUERY SELECT p.nombre_producto FROM PRODUCTO p
14     WHERE p.disponibilidad <> 'S';
15
16 -- Sales by date range (Date parameter fallback logic)
17 CREATE OR REPLACE FUNCTION fn_ventas_por_periodo(
18     p_fecha_inicio DATE, p_fecha_fin DATE DEFAULT NULL) /*...*/
19
20     IF p_fecha_fin IS NULL THEN p_fecha_fin := p_fecha_inicio;
21     END IF;
22     /*...*/
23     WHERE DATE(fecha_hora) BETWEEN p_fecha_inicio AND p_fecha_fin
24     ;

```

Listing 9: Core logic for Inventory and Sales Reporting

Implementation Logic: For the best-selling dish, we utilized a subquery inside the JOIN clause, applying `ORDER BY total_vendido DESC LIMIT 1` to dynamically isolate the most popular item. For the sales period function, we implemented a parameter fallback mechanism: if the user only provides a start date (`p_fecha_fin DEFAULT NULL`), the PL/pgSQL block automatically assigns the start date to the end date. This allows the

same function to seamlessly handle both single-day queries and broad date-range scans using the `BETWEEN` operator.

4.6 Part Two: Data Ingestion and Error Orchestration

The second major objective of the project required orchestrating the insertion of daily operations from plain text files into the database while gracefully handling potential errors.

```
1 CREATE OR REPLACE PROCEDURE pr_cargar_ordenes_desde_txt(  
    p_ruta_archivo TEXT)  
2 LANGUAGE plpgsql AS $$  
3 DECLARE  
4     -- Variable declarations omitted for brevity  
5 BEGIN  
6     -- 1. Create a temporary staging table  
7     CREATE TEMPORARY TABLE IF NOT EXISTS staging_txt_lineas (  
        linea_completa TEXT) ON COMMIT DROP;  
8  
9     -- 2. Bulk load data from physical file  
10    EXECUTE format('COPY staging_txt_lineas FROM %L WITH (FORMAT  
    text);', p_ruta_archivo);  
11  
12    -- 3. Row-by-row processing and error trapping  
13    FOR v_registro IN SELECT regexp_split_to_array(linea_completa  
        , ',') AS datos FROM staging_txt_lineas  
14    LOOP  
15        BEGIN  
16            -- Parsing and Insertion logic here...  
17            INSERT INTO ORDEN (fecha_hora, total_pagar,  
                rfc_cliente, numero_empleado)  
18            VALUES (...);  
19            COMMIT;  
20        EXCEPTION WHEN OTHERS THEN  
21            GET STACKED DIAGNOSTICS v_error_msg = MESSAGE_TEXT;  
22  
23            -- Log the specific error without crashing the entire  
                batch  
24            INSERT INTO LOG_ERRORES_CARGA (folio_afectado,  
                descripcion_error)  
25            VALUES ('LINEA FALLIDA', v_error_msg);  
26            ROLLBACK;  
27        END;  
28    END LOOP;  
29 END;  
30 $$;
```

Listing 10: Stored Procedure for TXT Data Ingestion (Snippet)

Implementation Logic: Building a robust Extract, Transform, Load (ETL) process directly in PostgreSQL required an advanced Stored Procedure architecture. First, the procedure utilizes the `COPY` command to rapidly ingest the raw text data into a `TEMPORARY TABLE` (*staging_txt_lineas*). This acts as an isolated sandbox, ensuring the main tables

remain unaffected by malformed raw data. Second, the procedure iterates through the staging data using a `FOR` loop containing a nested `BEGIN ... EXCEPTION ... END` block. This is a crucial transaction management design: if a specific row contains invalid data (e.g., an incorrect data type or a missing Foreign Key), the `EXCEPTION` block catches the error, performs a `ROLLBACK` strictly for that iteration, logs the exact error details into a custom audit table (`LOG_ERRORES_CARGA`), and seamlessly continues processing the next line. This guarantees that one corrupted line in a text file does not abort the entire day's data load.

5 Presentation

For data flow orchestration and database connection, **Apache Airflow** was selected as the main tool, executed via the Windows Subsystem for Linux (WSL). This modality was chosen because it is the industry standard for creating data pipelines (ETL). Airflow allows scheduling and monitoring workflows through Directed Acyclic Graphs (DAGs), facilitating the connection to PostgreSQL through its official *Providers* and securely managing credentials.

5.1 Workflow Design (Pipeline)

The logical design for data ingestion from text files comprises the following automated phases:

1. **Extraction:** Scheduled reading of the `.txt` files generated at the end of the day, which contain operational information (orders, customers, product details).
2. **Transformation and Cleaning:** Parsing of text strings to ensure that date, currency, and foreign key formats match the intended relational schemas.
3. **Load:** Execution of `INSERT` statements into the transactional database, orchestrating the insertion order to respect the hierarchy of foreign keys (for example, registering the *Customer* before the *Order*).

5.2 Engine Validations and Robustness (Error Handling)

Although orchestration defines the flow, error handling and data validations do not depend exclusively on the Python script in Airflow. The database was designed with a fail-safe (*bulletproof*) architecture, shifting the responsibility for data integrity to the PostgreSQL engine.

Even if the orchestrator were to send corrupt or incomplete information from a text file, the database would reject the transaction, protecting the information through the following security layers:

- **Integrity Triggers:** Trigger functions that validate strict business rules before allowing an insertion or update (e.g., financial consistency in order billing).
- **Parameterized Functions:** Use of logical structures (`CASE WHEN` in queries and views that neutralize null values (`NULL`), converting them into safe financial zeros to prevent system crashes).

- **Domain Constraints:** Strict logical filters (such as `disponibilidad <> 'S'`) that prevent the insertion of unrecognized states into the inventory system caused by typographical errors in the text files.

In this way, the bulk insertion flow is doubly validated: first by exception handling (*try/catch*) in the Airflow DAG tasks, and definitively by the relational and algorithmic design of the database engine itself.

6 Conclusions

6.1 Avila Sanchez Elias

Participating in this project has been a highly enriching experience that bridged the gap between theoretical database concepts and practical implementation. My primary challenge was orchestrating the technical documentation and ensuring that the complex logic developed by the team—ranging from the Entity-Relationship Model to the advanced PL/pgSQL triggers and ETL procedures—was accurately and formally represented in LaTeX. Translating our structural decisions into a cohesive, professional English report required a deep, holistic understanding of every module in the system.

A significant success was successfully compiling the document to strict academic standards, effectively articulating our architectural choices, such as the subclass mapping for employees and the B-Tree index optimization. Ultimately, this project solidified my understanding of how a robust, well-documented relational database serves as the critical backbone for digitizing real-world business operations, and highlighted the importance of clear technical communication in software engineering.

6.2 Brena de León Víctor Javier

The development of this project made it possible to comprehensively apply the concepts studied throughout the course, taking them from theory into a concrete functional solution. The relational model built reflects a normalized design that respects referential integrity across the different entities of the system, covering everything from staff management and their specializations to order and product registration.

The incorporation of PL/pgSQL as a procedural extension allowed business rules to be applied directly in the database, which would otherwise depend on external application layers.

The implementation of functions and triggers proved particularly important for ensuring data consistency in system operations. Through these tools it was possible to validate conditions before an operation was confirmed, automatically recalculate derived values, and encapsulate complex queries into reusable objects that simplify access to information. The views and query functions built on the model allow the same logic to be reused without rewriting it each time it is needed. Together, these decisions show that a well-structured database does not just store data, it also helps keep the system organized and reliable.

6.3 Espinosa González Sara Sofía

During the initial stages of development, it was necessary to carefully dissect the restaurant's operational workflows and map out how real-world entities interact with one an-

other. This early phase proved quite difficult to manage, as handling abstract business logic, such as defining complex employee hierarchies and structural inheritance rules, required multiple iterations to ensure no vital details were lost. Managing this complexity was critical, as any oversight would have severely compromised the subsequent normalization and scripting phases. Furthermore, managing the integration of various technical pieces into a cohesive, polished final document demanded strict organization to maintain an academic and highly professional presentation.

Overcoming these initial modeling hurdles and refining the presentation layout was an intensive process that required constant communication with the entire team. Thanks to this development, we practiced how to bridge the gap between abstract user needs and rigid technical design through teamwork, all while applying the knowledge learned in this subject.

6.4 Hernández Flores Alan Uziel

To develop this project, we had to go through all the stages of database development. First, we had to understand the client's requirements in order to convey it conceptually in an entity-relationship model.

During the development of the Relational Model, it was necessary to take into account both the client's vision and the future processing of the database, assigning the most appropriate data types and foreign keys. And finally, we created code to generate tables, functions, triggers, views, and data inserts to achieve all the requested business rules.

Generating the code was the most time-consuming process, requiring research on how to implement certain actions, the most notable being the transfer of data through a text file.

Thanks to this development, we practiced the ability to develop projects from scratch through teamwork, all while applying the knowledge learned in this subject.

6.5 Palacio Amandi Fernando

To develop this project, our team navigated through every phase of the database life cycle. While my teammates focused on analyzing the client's requirements and structuring the theoretical models, my primary responsibility was the physical realization of the database using PostgreSQL.

During this implementation phase, I focused on writing the DDL scripts to create the tables, primary keys, and foreign keys, ensuring that all structural rules established during the design phase were hard coded into the engine. Additionally, I orchestrated the data population by designing DML scripts to insert real-world test scenarios. This population step was critical, as it allowed us to rigorously test the relational constraints, optimize physical storage with precise data types, and verify that the database natively blocks inconsistent operations.

Materializing the design and executing the system tests was an intensive process, requiring deep research into database programming to automate tasks like custom key sequencing and transactional calculation triggers. Thanks to this development, we successfully practiced how to build integrated, enterprise-ready systems from scratch through teamwork, all while applying the knowledge learned in this subject.